



Anlogic StateHoldIO_JTAG

上海安路信息科技股份有限公司

APUG_065 (v1.0) 2025 年 10 月



目 录

目 录	I
1 文档介绍	1
2 StateHoldIO_JTAG 介绍	2
2.1 应用背景	2
2.2 实现流程	2
2.3 实现原理	3
2.4 stateio_wrapper 模块	5
3 工程构造	7
3.1 Bypass 释放	8
3.2 Statehold_en_n 释放	8
4 软件流程	9
4.1 软件设置	9
4.2 生成 statehold svf 文件	9
4.3 生成背景升级 svf 文件	10
4.4 背景升级 flash	11
4.5 StateHold 模式下刷新 SRAM	11
4.6 释放 IO，由内部逻辑接管	11
4.7 切换至用户设计	12
4.8 SVF 文件合并	12
4.9 JTAG 级联	12
5 工程验证	13
5.1 OUTPUT 类型 IO	13



5.2 INOUT 类型 IO	15
6 AJE 平台验证.....	17
6.1 移植步骤	17
6.2 AJE 验证.....	17
7 用户自定义加载环境	18
8 注意事项	19
8.1 硬件注意事项	19
8.2 软件注意事项	19
附录 1 Flash 权限切换	21
附录 2 MSPI x2/x4 使用	23
X2/X4 模式使用.....	23
PLL 动态配置.....	23
EF3 PLL 寄存器.....	24
EF2 PLL 寄存器.....	25
动态配置接口	25
版本信息	27
免责声明	27



1 文档介绍

本文档主要介绍安路 EF2 系列、EF3 系列（不含 EF3L50/EF3L70/EF3LA0）、EF4 系列器件 IO 保持功能的实现方法，结合 TD 在线工具、嵌入式配置平台，构造相关例程，为用户评估在 EF2/EF3/EF4 器件实现 StateHoldIO_JTAG 功能提供参考。

此文档适用器件：

- EF2 系列；
- EF3 系列：仅限于 EE3L15/45, EF3L40/EF3L90（下文中出现的 EF3 仅代表这些器件）；
- EF4 系列；

对于 EF3 系列 EF3L50/70/A0 及其他支持 IO 保持功能的器件详见《APUG104_Anlogic StateHoldIO 解决方案》。

2 StateHoldIO_JTAG 介绍

2.1 应用背景

在 FPGA 应用设计中，对于已量产的设备，通常都会有远程升级的需求，比如通过 CPU 升级（.aje 文件）升级。安路科技提供了背景升级模式以满足客户的设计需求。在某些升级场景中，比如服务器、存储，由于某些 IO 与外部的电源、CPU 有耦合关系，要求在升级的过程中，其 IO 状态保持不变，在 FPGA 进入用户模式后，将 IO 状态切换至用户逻辑接管，以确保在升级过程中，IO 不影响到其他系统，而导致系统死机、重启。在 EF2/EF3/EF4 器件，针对此需求推出 StateHoldIO_JTAG 功能。

2.2 实现流程

使用 StateHoldIO_JTAG 功能的流程一般如下图所示，其中 xxx_stateio_xxx.svf 与 xxx_stateio_xxx_bypass.svf 的生成使用详见第 4 章。

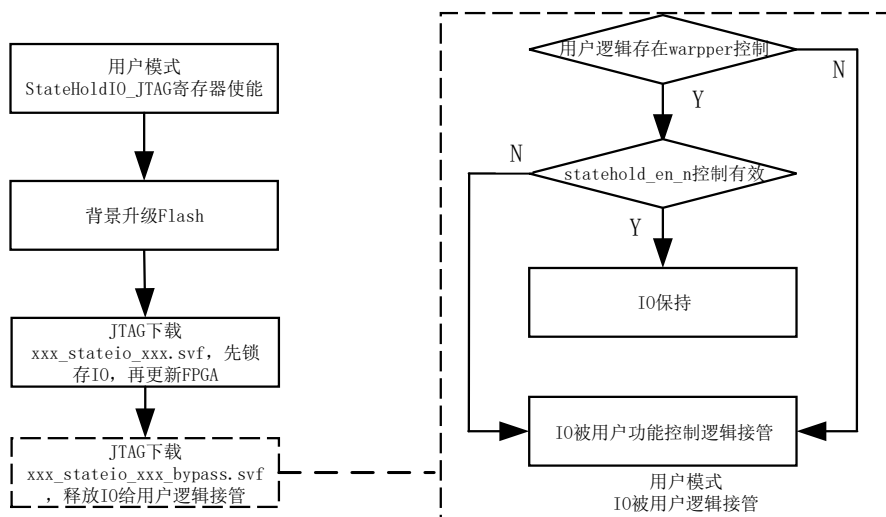


图 2-1 StateHoldIO_JTAG 功能使用流程

step1: 运行中的位流需使能 StateHoldIO_JTAG 相关寄存器；

step2: 在线升级时，一般会先通过 JTAG 背景模式更新 Flash 中的位流（更新完成后不会从 Flash 启动），此时 FPGA 正常运行。本步骤与 IO 保持功能无关，可在在线升级完成后再更新 Flash；

step3: 下载 xxx_stateio_xxx.svf，此 SVF 会先锁定除 JTAG IO 外的所有输出 IO；然后擦除 FPGA 中的 Case1（当前运行版本）并更新 Case2（待更新的版本）。配置过程及配置完成后，IO 状态将保持。

step4: 进入用户模式后，下载 xxx_stateio_xxx_bypps.svf，将 IO 释放给用户逻辑控制。

器件本身支持的 StateHoldIO_JTAG 功能，能够控制输出 IO 统一锁定、释放。若用户需要单独控制部分 IO 的锁定、释放，则可以通过在用户逻辑和 IO 之间增加 stateio_warpper 模块来实现。StateHoldIO_JTAG 功能释放 IO 时，stateio_warpper 模块能够接管 IO 状态控制，通过输入使能



statehold_en_n 控制 IO 状态继续保持或释放。模块介绍详见第 2.4 节。

2.3 实现原理

StateHoldIO_JTAG 实现的 IO 保持功能，其原理主要是通过 JTAG 回读 IO 状态，然后将 IO 状态下发并通过器件内部配置电路实现状态锁定，最后通过 JTAG 指令控制配置电路释放 IO 给用户逻辑。

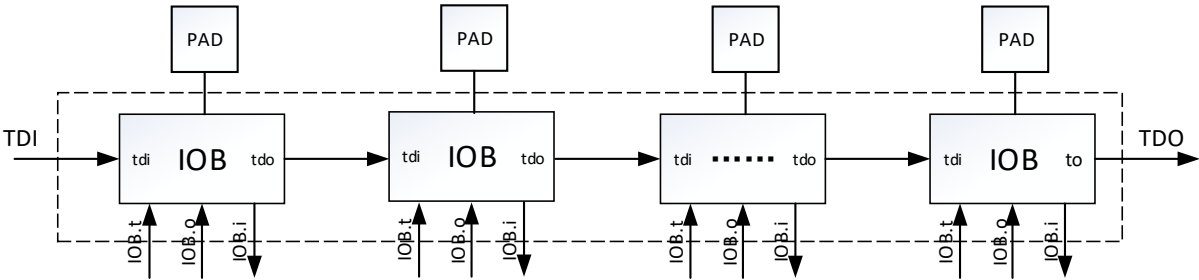


图 2-2 EF2/EF3/EF4 器件 IOB 结构框图

EF2/EF3/EF4 器件 IOB 的简易框图，如上图所示，其中 IOB.t、IOB.o、IOB.i，分别为 IOB 的输入方向控制、输出、输入。JTAG 可以串行访问所有 IO 状态，并通过内部配置电路实现 IO 的锁定。

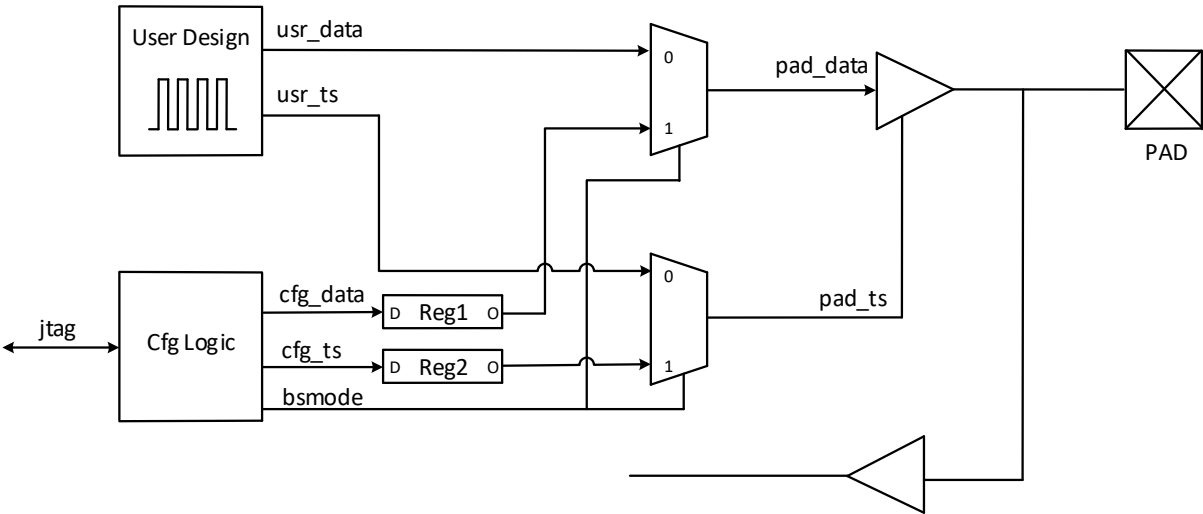


图 2-3 EF2/EF3/EF4 IO 结构框图

表 2-1 EF2/EF3/EF4 IO 结构信号说明

Name	Description
User Design	用户驱动 IO 的逻辑设计；
jtag	器件 JTAG 接口；
Cfg Logic	配置模块驱动 IO 的逻辑设计；
usr_data	用户驱动 IO 输出值；
usr_ts	用户驱动 IO 方向选择；
cfg_data	配置模块驱动 IO 输出值；



Name	Description
cfg_ts	配置模块驱动 IO 方向选择，IO 保持时为 0（输出方向）；
Reg1、Reg2	配置模块驱动 IO 的值及方向选择信号，都经 Reg 寄存后输出；
bsmode	IO 输出选择信号，由配置模块控制，IO 保持时 bsmode 为 1；
pad_data	输出至 PAD 的值；
pad_ts	PAD 的方向选择；
PAD	芯片接口；

单个 IO 内部结构的简易框图及信号说明分别如上图、上表所示。StateHoldIO_JTAG 功能实现过程中：

step1: 寄存器使能

正在运行的 Case1 需要使能 StateHoldIO_JTAG 功能，此时 FPGA 处于用户模式（bsmode=0），IO 状态由用户逻辑控制。

注意软件需要打开 stateholdio 选项，才能保证 IO 锁定后能够正确更新 FPGA。

step2: 背景更新 Flash

IO 保持、释放流程中只更新 FPGA，掉电易失。故在线升级时，一般会先通过 JTAG 背景模式更新 Flash 中的位流（Flash 读写权限需在配置电路侧），更新完成后不会从 Flash 启动，此时 FPGA 仍正常运行。本步骤与 StateHoldIO_JTAG 功能无关，在 FPGA 更新前或更新后都可以进行 Flash 更新。

step3: IO 锁定与更新 FPGA

对于已运行中的 Case1（正在运行的版本），下载 xxx_stateio_xxx.svf 进行 IO 锁定与 FPGA 更新，让 Case2（待更新的版本）进入用户模式工作，配置过程及配置完成后，IO 状态将保持。其中包含以下三步：

1. 首先通过 preload + extest 指令，回读 IO 状态，再将 IO 值与方向信号写入 Reg 中、bsmode 置 1，cfg_ts 置 0，以实现 IO 锁定；
2. 然后通过 erase + jtag burst 指令，擦除 FPGA 中的 Case1 并更新 Case2；
3. 最后通过 clamp +wakeup 指令，让 Case2 进入用户模式，此时 FPGA 内部逻辑正常工作，IO 状态仍保持。

step4: IO 释放

进入用户模式后，下载 xxx_stateio_xxx_bypss.svf，通过 Bypass 指令，将 bsmode 置 0，释放 IO 锁定，将 IO 的输出交给用户逻辑控制。

step5: 独立控制部分 IO 释放

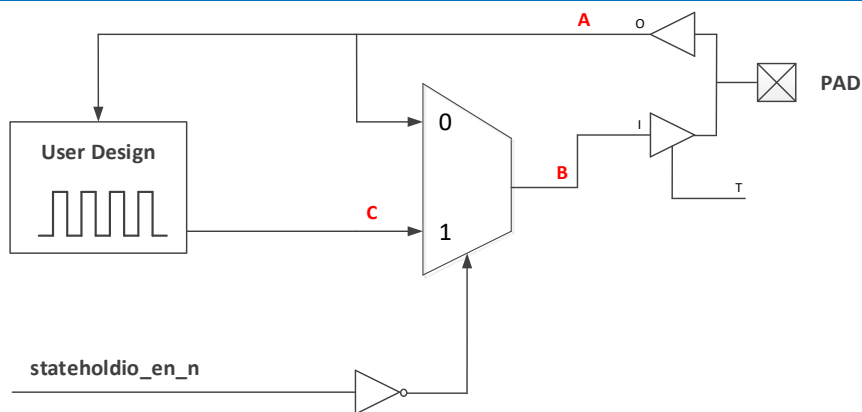


图 2-4 stateio_warpper 模块实现框图

用户可以使用 stateio_warpper 模块来控制双向 IO 单独释放，则实现原理框图如图 2-4 所示。其在用户逻辑和 IO 之间增加了 MUX，其中 stateholdio_en_n 为 mux 切换使能，当为 0 时，切换到用户逻辑；当为 1 时，锁存之前的反馈，保持其 IO 输出不变。stateio_warpper 模块介绍见 2.4 节。

2.4 stateio_warpper 模块

在使用 StateHoldIO_JTAG 功能时，若 IO 需要通过用户逻辑控制释放，则可以调用 stateio_warpper 模块，其接口及参数定义如下表所示：

表 2-2 state_warpper 模块接口及参数定义

Name	Type	Description
DATA_SIZE	参数	使用state_warpper模块处理的信号位宽；
HOLD_Z_EN	参数	state_warpper模块处理的信号类型 0：适用于OD输出的信号 1：适用于push-pull输出的信号
statehold_en_n	信号	Input, mux切换使能 0：释放IO，切换至用户逻辑接管 1：保持IO的状态不变
user_mux_din	信号	Input, 用户逻辑输入端口
stateio	信号	Inout, statehold io端口，连接至IO

源代码如下：



```
`timescale 1ns / 1ns

module stateio_wrapper

#(parameter

    DATA_SIZE = 8 ,

    HOLD_Z_EN = 0 //set hold state:1:output z; 0:hold state_input

)(

    input                                statehold_en_n , //0:user ctrl; 1:hold state

    input [DATA_SIZE-1:0] user_mux_din ,

    inout [DATA_SIZE-1:0] stateio

);

wire [DATA_SIZE-1:0] mux_latch;

wire [DATA_SIZE-1:0] fb_signal;

//Mux Latch

assign mux_latch = ~statehold_en_n ? user_mux_din : fb_signal;

//Set Direction

wire [DATA_SIZE-1:0] gpio_h1_en ;

genvar i;

generate

for (i = 0; i < DATA_SIZE; i=i+1)

begin:GENZ_LOOP

assign gpio_h1_en[i] =statehold_en_n&HOLD_Z_EN&stateio[i];

assign stateio[i]    = gpio_h1_en[i] ? 1'bz :mux_latch[i];

end

endgenerate

assign fb_signal = stateio ;

endmodule
```

3 工程构造

以 EF3L90 器件为例，在 EF3L90CG400B_MINI_V2.0 Demo 板上进行测试。板载时钟为 25 MHz，产生一个计数器，产生一个逻辑版本起来 7s 后的使能信号，和待送至 mux 端口的信号做&操作。按照图 2-4 的原理框图，在 IO 和用户设计中，加入 stateio_wrapper 模块。

```
12 //-----
13 // User Design Begin*****
14 // input clock : 25 MHz @ EF3L90CG400B_MINI_V2.0 Demo Board
15 //-----
16
17 reg [31:0] cnt_enable = 32'd0;
18 always @(posedge fpga_clk)
19 begin
20     if(cnt_enable == 32'hFFFF_FFFF)
21         cnt_enable <= cnt_enable;
22     else
23         cnt_enable <= cnt_enable + 1'b1;
24 end
25
26 //(hex2dec('AFF_FFFF'))/25e6 = 7.3s
27 reg user_design_enable;
28 always @(posedge fpga_clk)
29 begin
30     user_design_enable <= (cnt_enable >= 32'h0AFF_FFFF) ? 1'b1 : 1'b0;
31 end
32
33 reg [31:0] cnt = 32'd0;
34 always @(posedge fpga_clk)
35 begin
36     cnt <= cnt + 1'b1;
37 end
38
39 //push_pull test
40 reg to_mux_signal;
41 always @(posedge fpga_clk)
42 begin
43     to_mux_signal <= cnt[27] & user_design_enable;
44 end
45 assign test_io = to_mux_signal;
46
47 //OD test
48 wire to_mux_signal_z;|
49 assign to_mux_signal_z = (~user_design_enable) ? cnt[27] : 1'bz;
50 assign test_z_io = to_mux_signal_z;
51
52 //-----
53 // User Design End*****
54 // input clock : 25 MHz @ EF3L90CG400B_MINI_V2.0 Demo Board
55 //-----
56
57 wire statehold_en_n = state_online & statehold_en_n_user;
58 // 1 bit stateio_warpper
59 stateio_wrapper #(
60     .DATA_SIZE      ( 1 ),
61     .HOLD_Z_EN      ( 0 )//set hold state:1:hold z when input_state is 1; 0:hold state_input
62 ) u_stateio_wrapper_sig0(
63     .statehold_en_n  (statehold_en_n),
64     .user_mux_din    (to_mux_signal),
65     .stateio         (stateio_warpper_test_io)
66 );
67
68 // 1 bit stateio_warpper
69 stateio_wrapper #(
70     .DATA_SIZE      ( 1 ),
71     .HOLD_Z_EN      ( 1 )//set hold state:1:hold z when input_state is 1; 0:hold state_input
72 ) u_stateio_wrapper_sig1(
73     .statehold_en_n  (statehold_en_n),
74     .user_mux_din    (to_mux_signal_z),
75     .stateio         (stateio_warpper_test_z_io)
76 );
77
```



示例工程的接口信号及定义如下表所示：

表 3-1 示例工程接口及定义

Name	IO	Description
fpga_clk	INPUT	工作时钟，25MHz；
state_online	INPUT	上电启动、在线升级指示 0：当前为上电启动 1：当前为在线升级
statehold_en_n_user	INPUT	逻辑控制IO保持、释放使能，与state_online共同决定stateio_warpper模块 0：释放IO，切换至用户逻辑接管 1：保持IO的状态不变
test_io	OUTPUT	测试信号1，JTAG指令直接释放；
stateio_warpper_test_io	INOUT	测试信号2，stateio_warpper模块控制释放；
test_z_io	INOUT	测试信号3，INOUT类型，JTAG指令直接释放；
stateio_warpper_test_z_io	INOUT	测试信号4，INOUT类型，stateio_warpper模块控制释放；

3.1 Bypass 释放

对于未经过 stateio_warpper 模块的 IO，Bypass 指令生效后，IO 将直接释放。工程构造中包括 OUTPUT 类型的信号 test_io，与 INOUT 类型的信号 test_z_io。

3.2 Statehold_en_n 释放

对于经过 stateio_warpper 模块的 IO，Bypass 指令生效后，将用户逻辑产生 statehold_en_n 信号控制 IO 释放。工程构造中包括 INOUT 类型的信号 stateio_warpper_test_io 与 stateio_warpper_test_z_io。

注意使用 stateio_warpper 模块时有上电启动与在线升级两种加载场景，可以通过外部 IO 来指示。上电启动时通常不需要 IO 保持，将 state_online 置 0，IO 直接由用户逻辑控制；在线升级过程中，将 state_online 置 1，IO 由 statehold_en_n 控制锁定、释放。

4 软件流程

4.1 软件设置

生成 bit 文件前，需要设置 stateholdio 属性为 ON，且 boot mode 设置成 MSP1X1 模式，如图 4-1、4-2 所示。

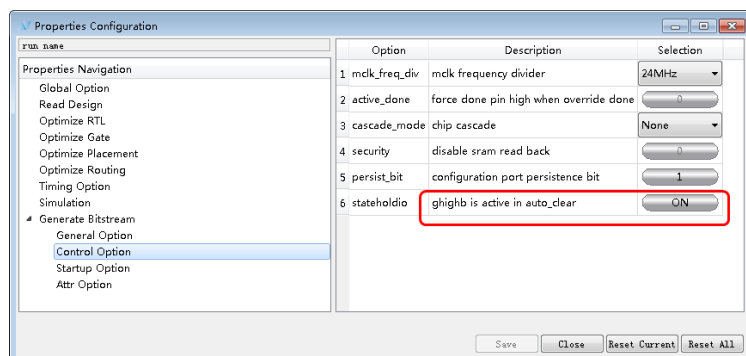


图 4-1 stateholdio 设置

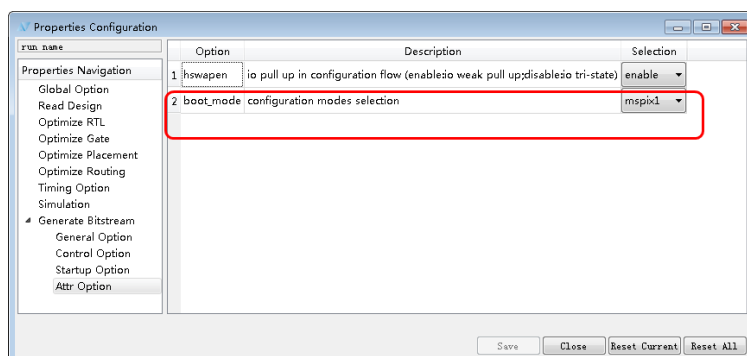


图 4-2 boot_mode 设置

4.2 生成 statehold svf 文件

在 TD 中选择 Tools-Device Chain，点击 Add 按钮，添加生成的 bit 文件；

点击 Create SVF 按钮，按照如下选项生成 SVF 文件，勾选 Enable State Hold IO，取消勾选 Use Non Burst，勾选 Verify 以添加回读校验功能，如图 4-3 所示。

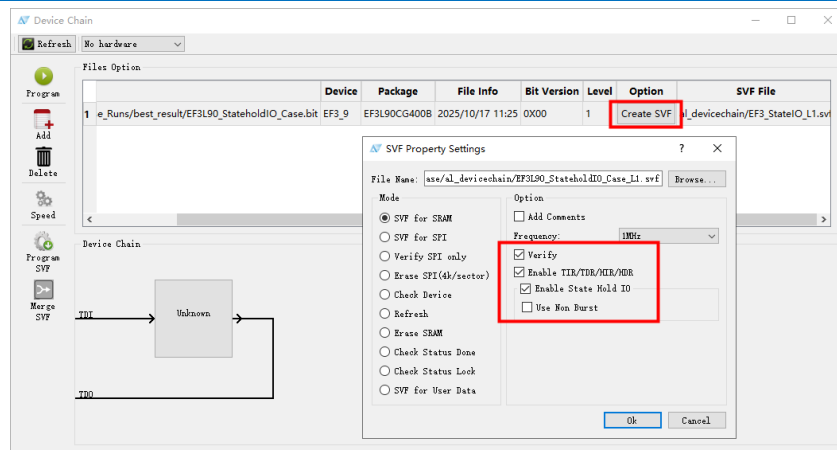


图 4-3 StateHoldIO_JTAG SVF 文件生成

在所选文件夹中将会生成 xxx_stateio_xxx.svf 和 xxx_stateio_xxx_bypass.svf 文件，如图 4-4 所示。

名称	修改日期	类型	大小
EF3_StateIO_L1.svf	2025/10/21 17:54	SVF 文件	2,405 KB
EF3_StateIO_L1_bypass.svf	2025/10/21 17:54	SVF 文件	1 KB

图 4-4 Statehold 相关文件

4.3 生成背景升级 svf 文件

在生成 SVF for SPI 时，勾选 bg_mode，不勾选 rf_mode，勾选 Verify 以添加回读校验功能，生成待升级的背景升级 Flash 文件，如图 4-5 所示。

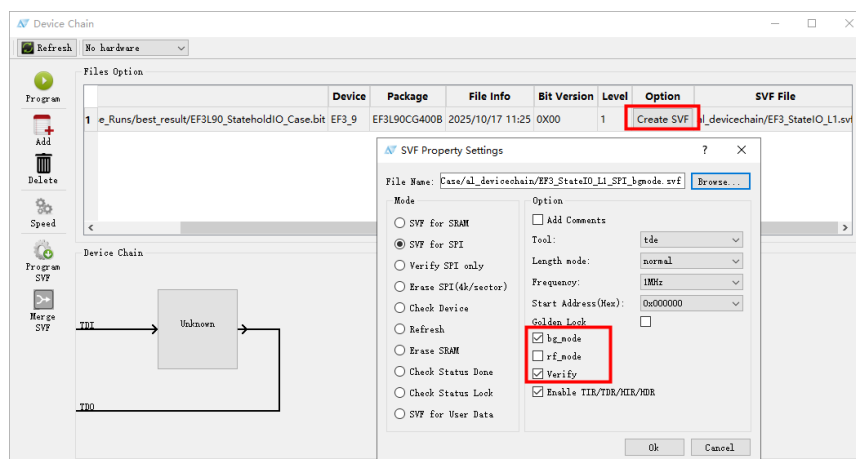


图 4-5 背景模式更新 Flash 的 SVF 生成

注：

1. 对于 bgmode 升级，persist bit 须设置成 1，否则升级无效。

4.4 背景升级 flash

打开 Download 工具，Mode 选择 JTAG，点击 Add，添加步骤 3 生成的 SVF 文件（Add 界面的“文件名”右边，可以选择文件类型为 svf file），点击“RUN”按钮进行加载，此步完成背景烧录 flash，如图 4-6 所示。

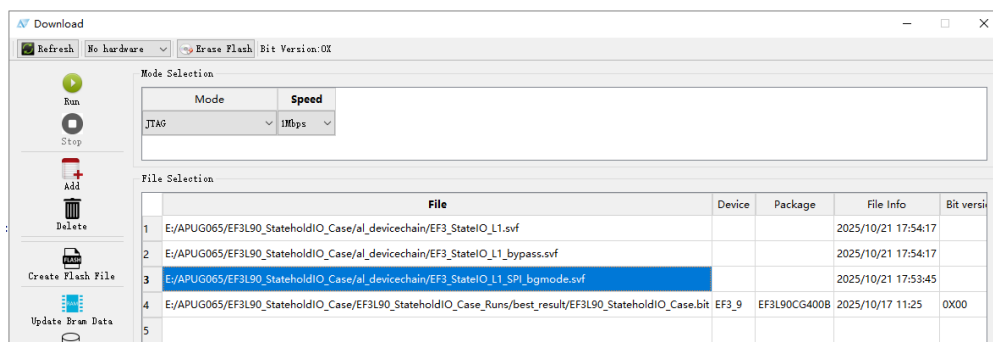


图 4-6 背景模式更新 Flash 的 SVF 下载

4.5 StateHold 模式下刷新 SRAM

点击 Add，添加步骤三生成的 **XX_StateIO_XX.svf** 文件，选中此 SVF 文件，点击“RUN”按钮进行加载，此步完成 sram 的刷新和输出 IO 管脚状态锁定，如图 4-7 所示。

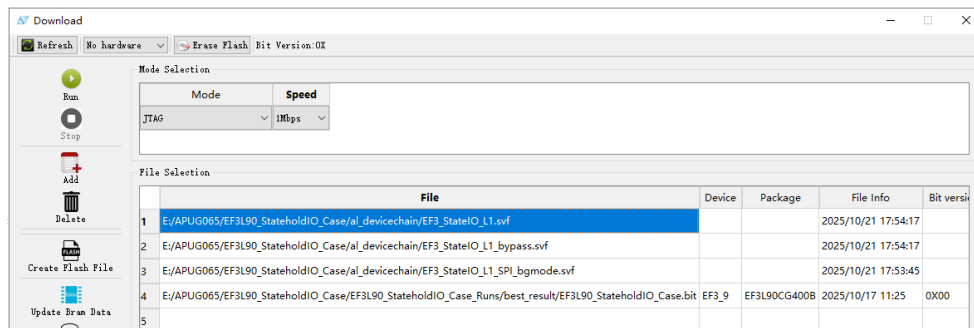


图 4-7 statehold sram 文件下载

4.6 释放 IO，由内部逻辑接管

点击 Add，添加步骤三生成的 **XX_StateIO_XX_bypass.svf** 文件，选中此 SVF 文件，当点击“RUN”按钮加载后，stateholdio 特性会释放掉 IO，由内部逻辑接管 IO 的输出状态，如图 4-8 所示。

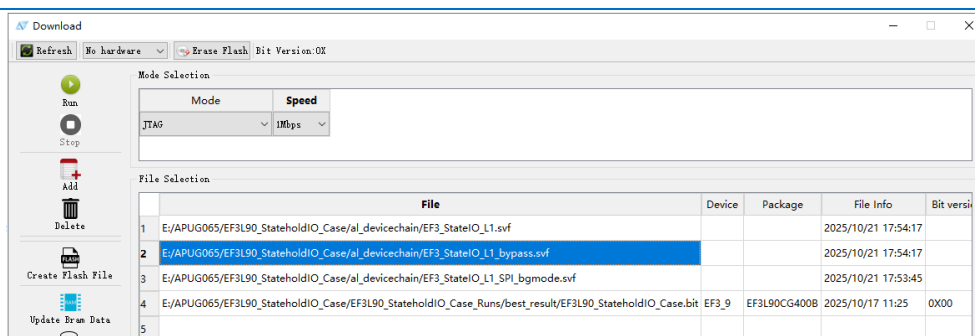


图 4-8 bypass 文件下载

4.7 切换至用户设计

对于使用 `stateholdio_en_n` 释放 IO 的场景，控制 `stateholdio_en_n` 为 0 后，将 IO 的驱动切换至用户输出逻辑驱动。

4.8 SVF 文件合并

在使用 `stateio_warpper` 模块来实现部分 IO 单独释放的场景中，通常可以将 `xxx_stateio_XXX.svf` 与 `xxx_stateio_XXX_bypps.svf` 进行合并、下载。使用 Merge SVF 工具先后添加 `xxx_stateio_XXX.svf` 与 `xxx_stateio_XXX_bypps.svf` 即可。

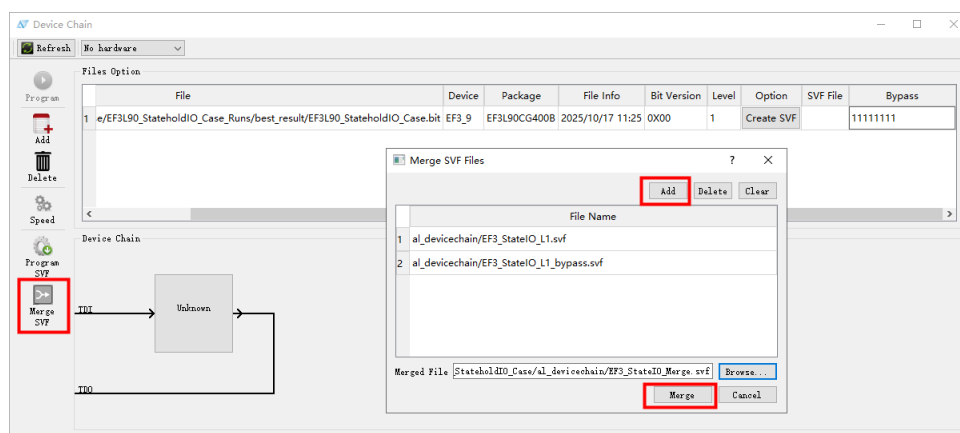


图 4-9 SVF 文件合并

4.9 JTAG 级联

若有级联需求，可通过 DeviceChain 工具，生成相对应的 SVF 文件。

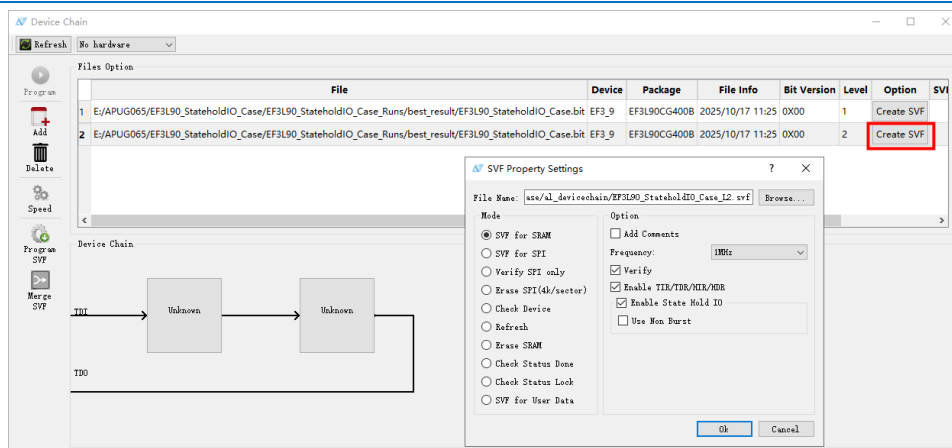


图 4-9 级联场景 SVF 文件生成

5 工程验证

5.1 OUTPUT 类型 IO

C1 波形为 test_io 信号，C2 波形为 stateio_warpper_test_io 信号。C2 为 INOUT 类型 IO，用户逻辑中只使用 OUTPUT 方向。

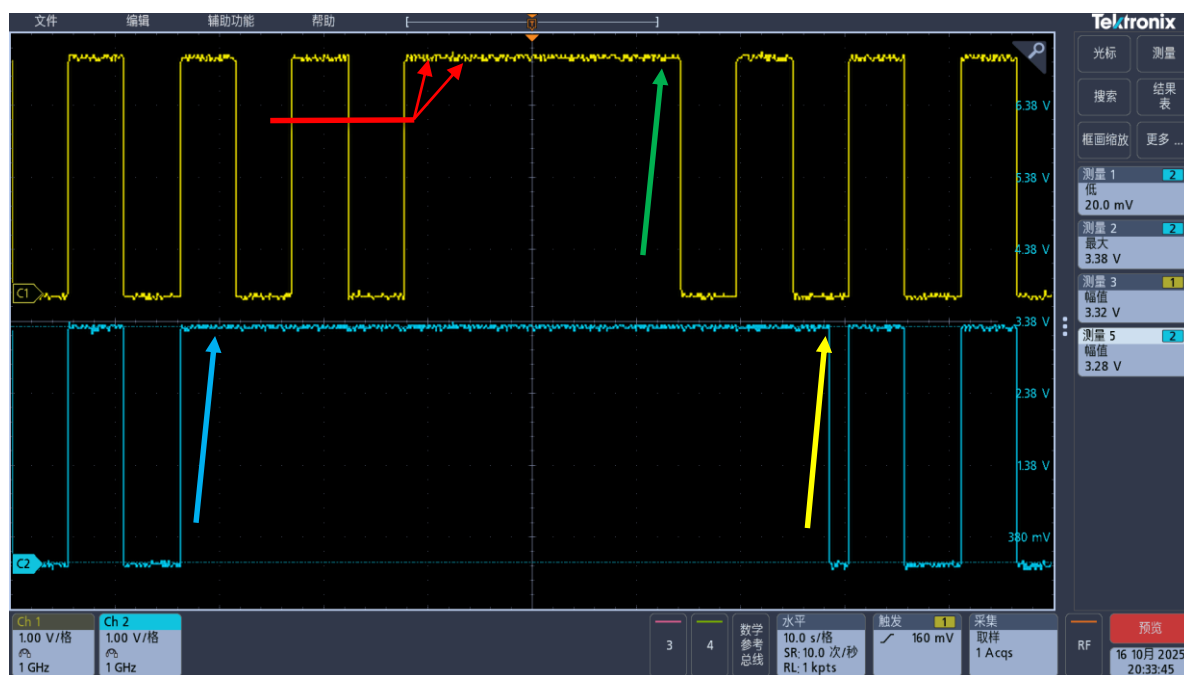


图 5-1 OUTPUT 类型 IO 保持高水平测试结果

过程如图 5-1 所示，描述如下：

1. 下载位流文件，进入用户逻辑后，将 state_online 置为 1；

2. 通过 `statehold_en_n` 信号 (Demo 对应的为按键输入, 低电平有效), 将 `stateio_warpper_test_io` 锁定在高电平, 如蓝色箭头所示;
3. 在红色箭头处下载 `EF3_StateIO_L1.svf`, 直到下载 `EF3_StateIO_L1_bypass.svf` 之前, IO 状态保持;
4. 下载 `EF3_StateIO_L1_bypass.svf`, 如绿色箭头指示, IO 被用户逻辑接管, 此时 `test_io` 正常翻转, `stateio_warpper_test_io` 继续保持当前高电平状态;
5. 当 `statehold_en_n` 信号拉低时, `stateio_warpper_test_io` 状态由高电平将切换到用户逻辑驱动, 如黄色箭头所示。

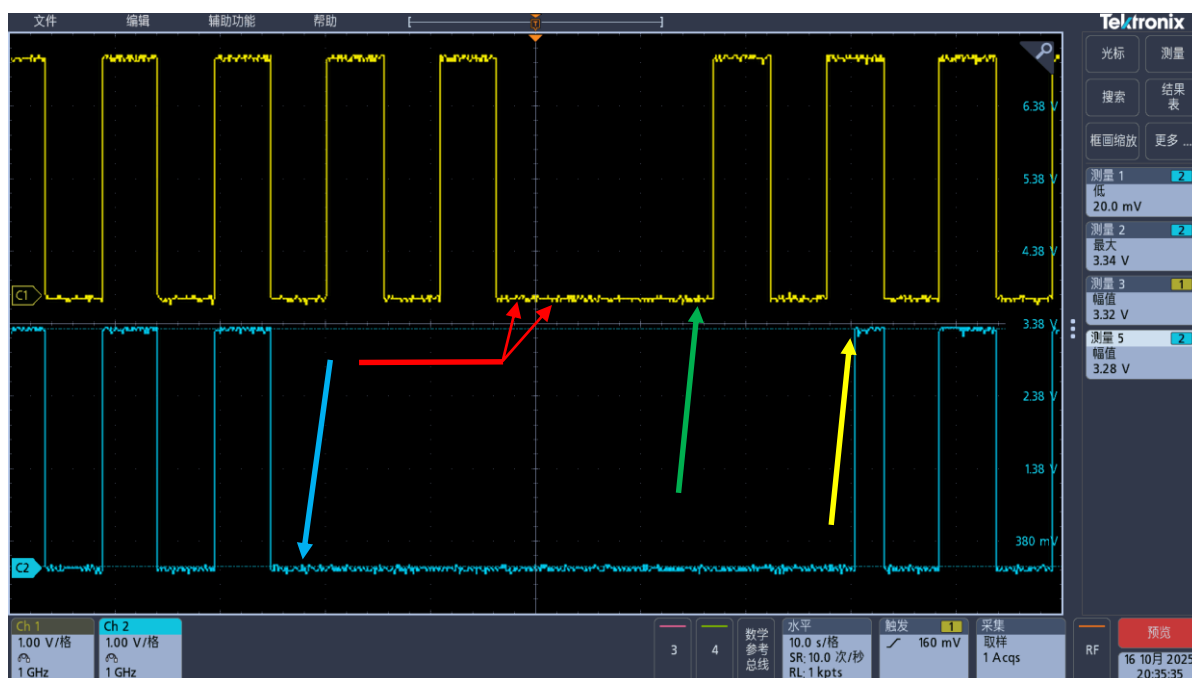


图 5-2 OUTPUT 类型 IO 保持低电平测试结果

过程如图 5-2 所示, 描述如下:

1. 下载位流文件, 进入用户逻辑后, 将 `state_online` 置为 1;
2. 通过 `statehold_en_n` 信号, 将 `stateio_warpper_test_io` 锁定在低电平, 如蓝色箭头所示;
3. 在红色箭头处下载 `EF3_StateIO_L1.svf`, 直到下载 `EF3_StateIO_L1_bypass.svf` 之前, IO 状态保持;
4. 下载 `EF3_StateIO_L1_bypass.svf`, 如绿色箭头指示, IO 被用户逻辑接管, 此时 `test_io` 正常翻转, `stateio_warpper_test_io` 继续保持当前低电平状态;
5. 当 `statehold_en_n` 信号拉低时, `stateio_warpper_test_io` 状态由低电平将切换到用户逻辑驱动, 如黄色箭头所示。

5.2 INOUT 类型 IO

C1 波形为 test_z_io 信号，C2 波形为 stateio_warpper_test_z_io 信号。C1、C2 在用户逻辑中均作为 INOUT IO。

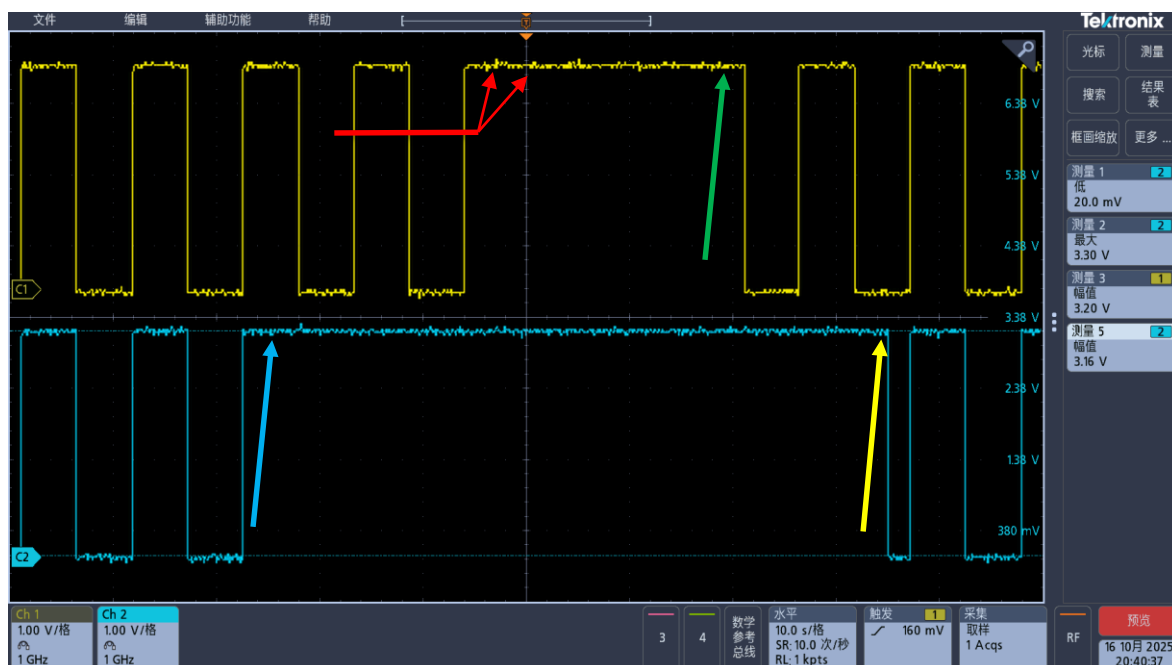


图 5-3 INOUT 类型 IO 保持高电平测试结果

test_z_io 与 stateio_warpper_test_z_io 信号均通过电阻下拉，如图 5-3 所示，下载位流文件过程描述如下：

1. 进入用户逻辑后，将 state_online 置为 1。test_z_io 与 stateio_warpper_test_z_io 信号在 7s 后方向均切换为 OUTPUT；
2. 通过 statehold_en_n 信号，将 stateio_warpper_test_z_io 锁定在高电平，如蓝色箭头所示；
3. 在红色箭头处下载 EF3_StateIO_L1.svf，直到下载 EF3_StateIO_L1_bypass.svf 之前，IO 状态保持；
4. 下载 EF3_StateIO_L1_bypass.svf，如绿色箭头指示，IO 被用户逻辑接管，此 test_z_io 正常翻转，stateio_warpper_test_z_io 继续保持当前高电平状态；
5. 当 statehold_en_n 信号拉低时，stateio_warpper_test_z_io 状态由高电平将切换到用户逻辑驱动，如黄色箭头所示。

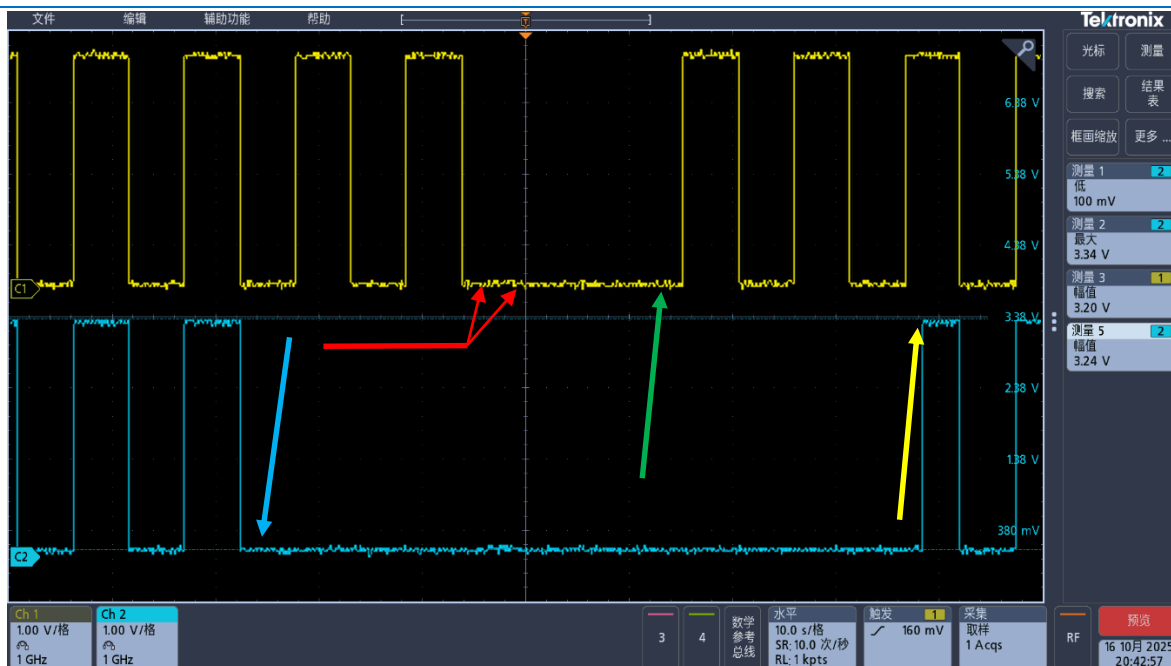


图 5-4INOUT 类型 IO 保持低电平测试结果

test_z_io 与 stateio_warpper_test_z_io 信号均通过电阻上拉，如图 5-4 所示，下载位流文件，过程描述如下：

1. 进入用户逻辑后，将 state_online 置为 1。test_z_io 与 stateio_warpper_test_z_io 信号在 7s 后方向均切换为 OUTPUT；
2. 通过 statehold_en_n 信号，将 stateio_warpper_test_z_io 锁定在低电平，如蓝色箭头所示；
3. 在红色箭头处下载 EF3_StateIO_L1.svf，直到下载 EF3_StateIO_L1_bypass.svf 之前，IO 状态保持；
4. 下载 EF3_StateIO_L1_bypass.svf，如绿色箭头指示，IO 被用户逻辑接管，此 test_z_io 正常翻转，stateio_warpper_test_z_io 继续保持当前低电平状态；
5. 当 statehold_en_n 信号拉低时，stateio_warpper_test_z_io 状态由低电平将切换到用户逻辑驱动，如黄色箭头所示。

6 AJE 平台验证

6.1 移植步骤

AJE 版本移植到嵌入平台，将 TMS/TCK/TDI/TDO 绑定到 CPU GPIO 上，有关 AJE 移植可参考《TN008 Anlogic AJE 移植指南》；

如果 TD 软件版本不能生成 AJE 文件，则可通过 svf2aje 工具，将生成 xxx_stateio_xxx.svf 和 xxx_stateio xxx_bypass.svf 等文件，通过 svf2aje.exe 转成相关 AJE 文件，其命令如下：

```
svf2aje.exe -svffile EF3 Statel0 L1.svf -ajefile EF3 Statel0 L1.aje
```

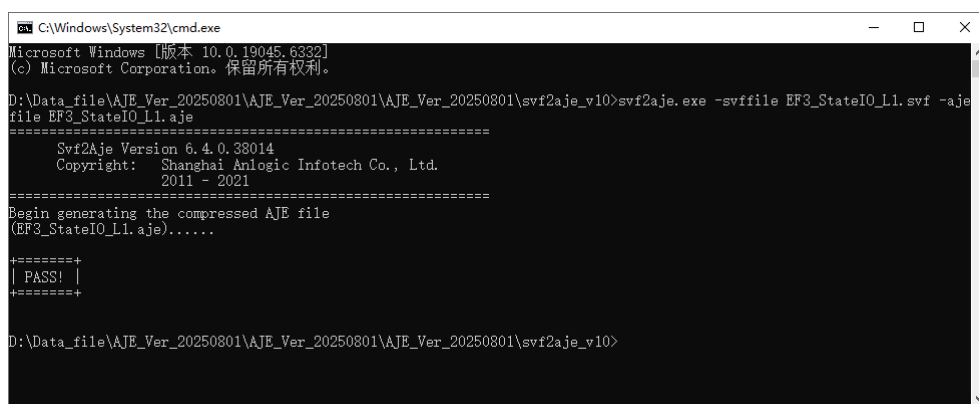


图 6-1 AJE 文件生成

6.2 AJE 验证

AJE 配置平台，下载流程和之前的操作步骤类似：先下载背景升级 Flash AJE 文件，然后再下载 stateio AJE 文件，刷新 SRAM，最后再下载 bypass AJE 文件。

在 CPU 通过 GPIO 模拟 JTAG 程序中，AJE 提供 RSDR 指令，通过 RSDR 可实现自动获取 IO 状态，格式如下：

RSDR		688		TDI
(ff000f00000f000000000000 00ff)				TDO
(ff000f00000f000000000000 00ff)				
MASK				
(ff ff) ;				

在 AJE 验证平台进行验证测试，调试信息如图 6-2 所示。

8 注意事项

8.1 硬件注意事项

1. PROGRAMN 作为复用 IO 时，不能使用 StateHoldIO_JTAG 功能；
2. 关于 INOUT 类型 IO，作输出时锁定固定电平，锁定电平是强驱；作输出时锁定高阻，需在 IO 锁定前，逻辑控制 IO 切换成输入。
3. 若使用 stateio_warpper 模块，推荐使用 GPIO 作为 state_online，以区分上电启动与在线升级场景，一般使用流程如下图所示。在上电启动的场景中，注意设置 state_online 为 0，启动时 IO 直接受用户逻辑控制；在线升级场景中，state_online 需为 1，升级完成后 IO 才能由 stateio_warpper 模块继续保持，直到用户逻辑控制释放。

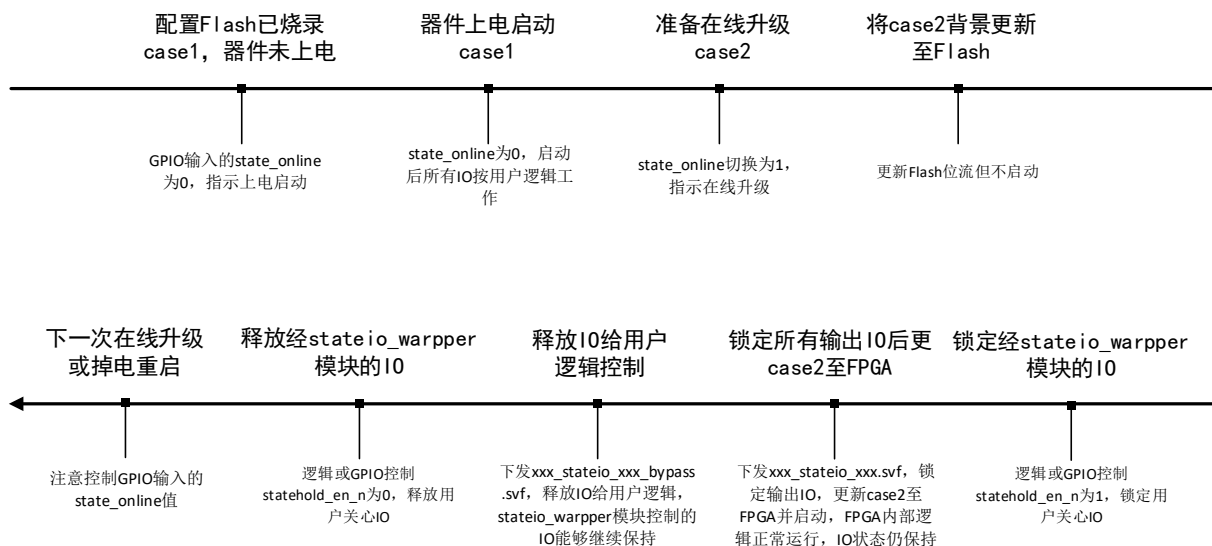


图 8-1 GPIO 作为 state_online 时 IO 保持流程

8.2 软件注意事项

1. 使用 StateholdIO_JTAG 功能时，如果只是通过 Bypass 指令释放 IO（适用于加载完成后立马释放 IO，或者等待一段固定时间后释放 IO 的场景），可以将使用 IO 保持特性的管脚定义为 OUTPUT 类型。Bypass 指令释放后需要通过 stateio_warpper 模块单独控制释放的管脚，则需要定义为 INOUT 类型。
2. 建议将 xxx_stateio_XXX.svf 与 xxx_stateio_XXX_bypps.svf 进行合并。
 - 加载完立马释放 IO 的场景，与需要通过 stateio_warpper 模块单独控制 IO 释放的场景，两个 SVF 直接合并、下载即可；
 - 加载完成后等待固定时长释放 IO 的场景，可通过手动在两个 SVF 中间插入 RUNTEST 指令的方

式实现，固定等待时长 = RUNTEST 个数 × TCK 周期，示例如下：

```
42 RUNTEST 100 TCK ;
43 SIR 8 TDI (3d) ;
44 RUNTEST 15 TCK ;
45 SIR 8 TDI (0A) ;
46 RUNTEST 15 TCK ;
47
48 // xxx_stateio_xxx.svf End
49 //-----
50
51 // IO StateHold Duration After FPGA Programing Completes
52 // Example: Duration = 20000*10e6 s(if TCK FREQUENCY=1MHz)
53 RUNTEST 20000 TCK ;
54
55 //-----
56 // xxx_stateio_xxx_bypss.svf Begin
57
58 // Created using Anlogic Software, 5.6.88061
59 // Date: 2025/10/17 11:25
60 // Architecture: ef3_9
61 TRST OFF;
62 ENDIR IDLE;
63 ENDDR IDLE;
64 FREQUENCY 1E6 HZ ;
65 TDR 0 ;
66 TIR 0 ;
```

图 8-2 手动添加固定等待时长方法示意

3. 对应 SVF 文件中的 RUNTEST 命令，其 TCK 必须翻转；
4. 在生成背景升级 Flash 的 SVF 文件时，需勾选 **bg_mode** 实现背景模式更新；勾选 **Verify** 以添加回读校验功能；不勾选 **rf_mode** 避免更新完成后从 Flash 启动，导致 IO 失锁；
5. 背景升级 Flash 时需确保 Flash 读写权限在配置电路侧。若用户逻辑不调用 **INTFLASH** 原语操作配置 Flash，则需要设置 **persist_bit=1**。若用户逻辑使用了配置 Flash，则需要参考附录 1，先收回 Flash 读写权限至配置电路、再进行背景升级；
6. 对于 EF2、EF3L15/45/40/90 系列器件，StateHoldIO_JTAG 功能实现时，位流需要设置为 **MSPIx1** 模式。生成 SVF for SRAM 文件时，勾选“**Enalbe State Hold IO**”，同时不勾选“**Use Non Burst**”，以生成 **burst** 类型的 SVF；

若器件正在运行的位流是以 **MSPIx2/x4** 方式加载启动成功的，此时 JTAG 无法通过 **burst** 方式升级 FPGA。故对 **MSPI** 加载速度有要求，需使用 **MSPIx2/x4** 加载模式的位流时，在生成 SVF for SRAM 文件时，勾选“**Enalbe State Hold IO**”，同时勾选“**Use Non Burst**”，以生成 **non-burst** 类型的 SVF。此时 PLL/RAM 等模块则需要进入用户模式后才能初始化，详见附录 2；

对于 EF4 系列器件，StateHoldIO_JTAG 功能实现时，用于生成 SVF 的位流，没有 **MSPIx1/x2/x4** 的位宽限制；

7. 本方案中，JTAG 先回读 IO 状态，再进行锁定，对于快速变化的信号需谨慎使用。

附录 1 Flash 权限切换

背景升级 Flash 时需确保 Flash 读写权限在配置电路侧。若需要在用户逻辑读写配置 Flash，则需调用 INTFLASH 原语、设置 `persist_bit=0`。这样 MSPI 加载位流完成后，Flash 的控制权会切换给用户逻辑使用。此时若想背景升级 Flash，则需按照以下步骤进行：

step1: 通过 JTAG 指令设置 `persist_bit=1`，将 Flash 权限切换至配置模块；

step2: 背景升级 Flash；

step3: 通过 JTAG 指令设置 `persist_bit=0`，将 Flash 权限切换至用户逻辑。

其中，step1 与 step3 中用于设置 `persist_bit` 的 SVF 文件生成方法如下（适用于 TD5.6.2_76111 及以后的版本）：

1. 在 TD console 窗口执行以下指令，生成设置 `persist_bit=1` 的 SVF 文件，其中 `{xxx\xxx.bit}` 为指定路径的用户位流，`{xxx\xxx_persist_set_1.svf}` 为生成到指定路径的 SVF 文件。

```
svfgen -scan_mode prog_register -bit {xxx\xxx.bit} -g {persist_bit:1} -svf  
{xxx\xxx_persist_set_1.svf}
```

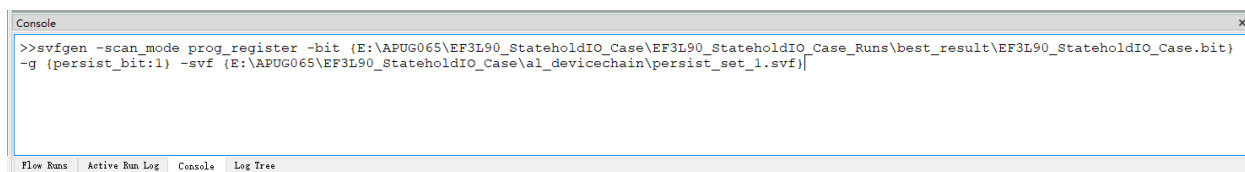
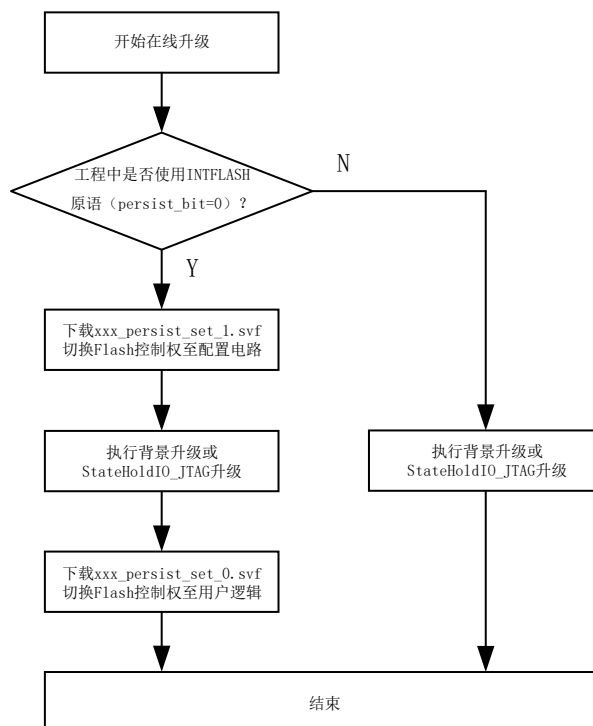


图 9-1 persist_bit 设置 SVF 生成示例

2. 在 TD console 窗口执行以下指令，生成设置 `persist_bit=0` 的 SVF 文件，操作步骤与（1）类似。

```
svfgen -scan_mode prog_register -bit {xxx\xxx.bit} -g {persist_bit:0} -svf  
{xxx\xxx_persist_set_0.svf}
```

综上，用户逻辑需要读写配置 Flash 的场景中，进行在线升级时的操作流程如下图所示：



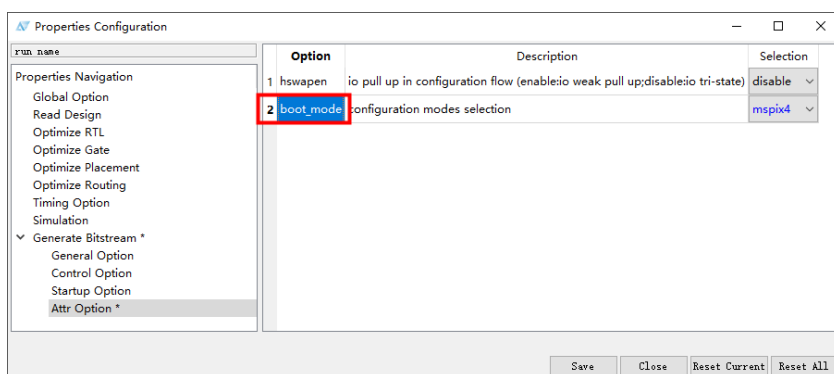
附录 2 MSPI x2/x4 使用

对于器件加载速度有要求，可考虑 X2/X4 模式；在此模式下，PLL/RAM 等模块需进入用户模式后才能通过动态配置的方式进行初始化。

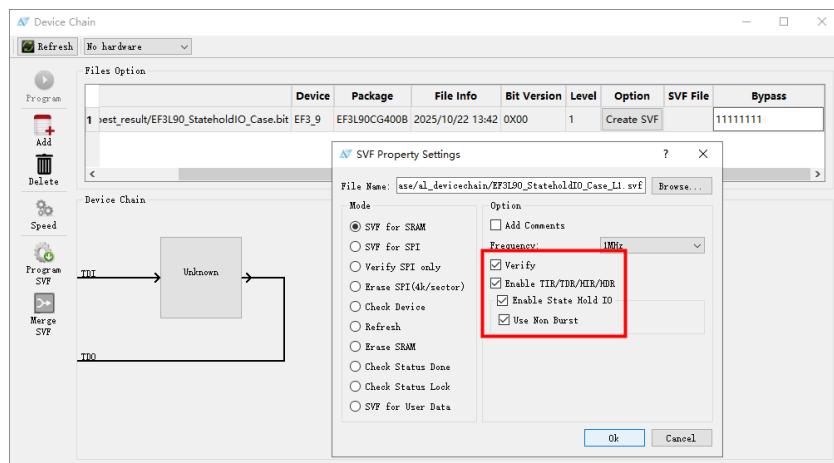
相关资料可参考《TN425_PLL 配置》、《TN604_EF3 PLL 模块用户指南》、《TN608_EF3 PLL 动态配置》。

X2/X4 模式使用

在 MSPIX2/X4 模式下，只能使用 non-burst 升级不包含 eram 的码流(即，ERAM(ram/pll)不能作初始化，PLL 需通过动态配置接口实现)。bootmode 配置如下图所示。



Non burst SVF 文件配置如下图所示。



PLL 动态配置

动态配置是用户可以在 PLL 工作过程中通过专门的输入输出接口直接控制锁相环的配置参数，包括：

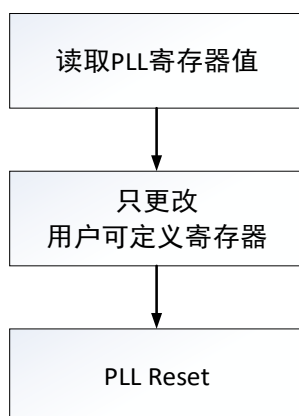
- 参考时钟分频系数 (M)；
- 反馈时钟分频系数 (N)；



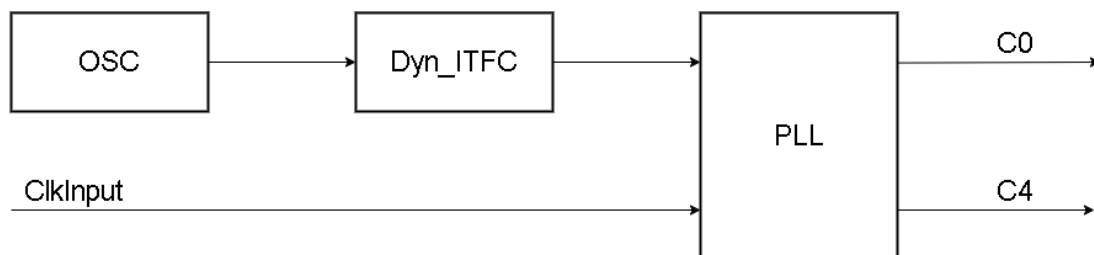
- 输出时钟分频系数 (C0~4)；
- 输出相位 P；

为了保证 PLL 动态配置达到预期，配置 PLL 相关寄存器需严格按照控制位列表进行，先读取后，只更新用户可定义寄存器值，最后再复位，确保配置成功。

PLL 动态配置流程如下，更多 PLL 动态配置，参见《TN608_EF3 PLL 动态配置.pdf》。



PLL 的每个参数都存储在 32*8 的 RAM 空间中。动态配置功能通过 AL_PHY_PLL 中的类似 MEMORY 访问的 8 位动态接口来实现。此应用以 PLL C0/C1 输出占空比为例，对 PLL 动态配置使用进行说明。在整个配置过程中，C0 和 C1 的占空比占据寄存器分布在各个寄存器的最高位上，PLL 动态配置框图如下：



EF3 PLL 寄存器

EF3 PLL 动态配置表如下：

地址	说明
5' h00	[7:2]=Channel0_MC1_DUTY[5:0]，[1:0]禁止用户改写；
5' h01	[7]=0，[6:0]=参考时钟分频系数；
5' h02	[7]=CHANNEL1_MC1_DUTY[2]，[6:0]=反馈时钟分频系数；
5' h03	[7]= CHANNEL1_MC1_DUTY[4]，[6:0]=通道 0 延迟系数；
5' h04	[7]= CHANNEL1_ MC1_DUTY[0]，[6:0]=通道 0 分频系数；
5' h05	[7]= CHANNEL1_MC1_DUTY[1]，[6:0]=通道 1 延迟系数；
5' h06	[7]= CHANNEL1_MC1_DUTY[5]，[6:0]=通道 1 分频系数；
5' h07	[7]= CHANNEL1_MC1_DUTY[6]，[6:0]=通道 2 延迟系数；



地址	说明
5' h08	[7]=0, [6:0]=通道 2 分频系数;
5' h09	[7]=0, [6:0]=通道 3 延迟系数;
5' h0A	[7]=0, [6:0]=通道 3 分频系数;
5' h0B	[7]=CHANEL1_MC1_DUTY50_N, [6:0]=通道 4 延迟系数;
5' h0C	[7]= CHANEL1_MC1_DUTY[3], [6:0]=通道 4 分频系数;
5' h11	[3]= CHANELO_MC1_DUTY[6], 其它位禁止改写;
5' h12	[2:0]=通道 0 相位, [5:3]=通道 1 相位, [7:6]=通道 2 相位的第 1、0 位;
5' h13	[0]=通道 2 相位 2 位, [3:1]=通道 3 相位, [6:4]=通道 4 相位, [7]=0;
5' h18	[4]=CHANELO_MC1_DUTY50_N, 其它位用户必须读出后再写回原值;

注:

1. 上述寄存器的值与 TD PLL GUI 界面值之差为 1, 即 GUI=20, 配置 Reg=19。

EF2 PLL 寄存器

EF2 PLL 动态配置寄存器定义如下:

地址	Bit[7:0]	说明
5' H01	Refclk_div[6:0]	参考钟分频系数, Div_Factor-1
5' H02	fdbkclk_div[6:0]	反馈时钟分频系数, Multi_Factor-1
5' H04	channel0_div[6:0]	通道 0 分频系数, 不可动态更新
5' H06	channel1_div[6:0]	通道 1 分频系数, 不可动态更新
5' H08	channel2_div[6:0]	通道 2 分频系数, 5-1
5' H0A	channel3_div[6:0]	通道 3 分频系数, 20-1
5' H0C	channel4_div[6:0]	通道 4 分频系数,

注:

1. EF2 PLL 动态配置数据更新只涉及到 C2~C4 div 以及参考钟、反馈钟的分频系数。

动态配置接口

PLL 动态配置接口一般如下:

Signals	I/O	Width	Description
dcIk	I	1	PLL 动态配置模块时钟输入, 上升沿有效
dcs	0	1	片选输入, 高有效, 时钟上升沿被锁存
dwe	0	1	写使能高有效, 时钟上升沿被锁存
di	0	8	写入数据
daddr	0	6	读写地址, 时钟上升沿被锁存



Signals	I/O	Width	Description
do	I	8	数据无延迟输出, PLL 配置数据读出端口



版本信息

日期	版本	修订记录
2025/10/21	1.0	首次发布中文版

版权所有© 2025 上海安路信息科技股份有限公司

未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止发言或其他方式授予任何知识产权许可；本文档仅为向用户提供使用器件的参考，协助用户正确地使用安路科技产品之用，其著作权归安路科技所有；本文档所展示的任何产品信息均不构成安路科技对所涉产品或服务作出任何明示或默示的声明或保证。

安路科技将不定期地对本文档进行更新、修订。用户如需获取最新版本的文档，可通过安路科技的官方网站（网址为：<https://www.anlogic.com>）自行查询下载，也可联系安路科技的销售人员咨询获取。